

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”
(УНИВЕРСИТЕТ ИТМО)

ЦЕНТР АВТОРИЗОВАННОГО ОБУЧЕНИЯ ИНФОРМАЦИОННЫМ ТЕХНОЛОГИЯМ

ИТОГОВАЯ АТТЕСТАЦИОННАЯ РАБОТА

Система автоматизации работы фитнес-клуба

Автор Харченко Дмитрий _____

(Фамилия Имя Отчество)

(Подпись)

Центр авторизованного обучения информационным технологиям

Наименование программы **«Python-разработчик»**

Руководитель Осипов Никита Алексеевич _____

(Подпись)

К защите допустить

Заместитель директора ЦАО ИТ, к.т.н. _____

/ Т.В. Зудилова/

Санкт-Петербург, 2026г.

Обучающийся _____ Группа 124/19
(Фамилия, И. О.)

Работа принята «__» _____ 2026г.

Работа выполнена с оценкой _____

Дата защиты «__» _____ 2026г.

Секретарь ИАК _____
(Фамилия, И. О.) (подпись)

Листов хранения _____

Демонстрационных материалов _____

Содержание

ВВЕДЕНИЕ	4
1 АНАЛИЗ CRM СИСТЕМ ДЛЯ ФИТНЕС-КЛУБА.....	7
1.1 Анализ деятельности фитнес-клуба	7
1.2 Выявление ролей пользователей и функциональных требований ...	8
1.3 Анализ существующих аналогов программного обеспечения	10
2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ	12
2.1 Модульная структура приложения	12
2.2 Объектно-ориентированное проектирование	12
2.3 Проектирование базы данных	13
3 РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ	16
3.1 Обоснование выбора типа приложения	16
3.2 Обоснование выбора технологий	17
3.3 Реализация структуры Django-проекта	18
3.4 Клиент-серверная архитектура приложения	19
4 ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ	20
4.1 Описание тестируемых компонентов	20
4.2 Проверка моделей и бизнес-логики	20
5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	21
5.1 Запуск приложения	21
5.2 Вход в административную панель	22
5.3 Работа с клиентами	23
5.4 Работа с тренерами	24
5.5 Работа с услугами и абонементом	24
5.6 Оформление абонемента клиента	24
5.7 Работа с занятиями и записями	25
5.8 Учет разовых посещений	26
5.9 Завершение работы	26
ЗАКЛЮЧЕНИЕ	27
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	29

Введение

В сфере услуг часть проблем связана не с самой услугой, а с организацией работы вокруг нее. Клиентов нужно учитывать, записи - не терять, расписание - поддерживать в актуальном состоянии, а абонементы - контролировать по срокам и количеству посещений. Для фитнес-клуба это особенно заметно: в одной системе должны быть связаны клиенты, тренеры, услуги, занятия, записи, разовые посещения и абонементы.

На первый взгляд такие задачи можно решать простыми средствами. Например, использовать Excel или другую электронную таблицу. На самом раннем этапе это действительно может работать. Но когда клиентов становится больше, появляются ошибки: данные начинают дублироваться, записи теряются, становится сложнее быстро понять, кто записан на занятие, сколько осталось мест, есть ли у клиента действующий абонемент.

Другой вариант - использовать универсальную CRM-систему. Такие системы удобны для продаж, сделок и коммуникации с клиентами, но они не всегда хорошо подходят под конкретную сферу услуг. Для фитнес-клуба важны не только контакты клиентов, но и расписание, тренеры, занятия, рабочие интервалы, абонементы и посещения. Если система изначально не рассчитана на такие процессы, ее приходится настраивать под себя, а это не всегда удобно.

Есть и специализированные решения для фитнес-клубов. Они ближе к нужной области, но у них есть другой недостаток - стоимость и зависимость от уже готовой логики продукта. Часто приходится использовать не то, что полностью подходит организации, а то, что доступно на рынке. В итоге получается компромисс: система лучше Excel, но не всегда полностью соответствует реальным процессам конкретного клуба.

Идея данной работы связана именно с этой проблемой. Возникла необходимость спроектировать простую и расширяемую систему, которая закрывает базовые задачи фитнес-клуба: учет клиентов, тренеров, услуг, занятий, записей, абонементов и разовых посещений. В текущей версии проект не претендует на полноценную коммерческую CRM-систему. Это скорее основа, которую можно развивать дальше.

Актуальность темы заключается в том, что даже небольшому фитнес-клубу нужна единая система учета. Без нее данные оказываются разрозненными: часть информации хранится в таблицах, часть - в переписках, часть - у администратора или тренера. Такой подход усложняет работу и повышает вероятность ошибок. Автоматизированная система позволяет хранить данные централизованно и быстрее выполнять основные операции.

Целью дипломной работы является разработка веб-приложения для автоматизации работы фитнес-клуба, обеспечивающего учет клиентов, тренеров, услуг, занятий, записей, разовых посещений и абонементов.

Запросы пользователей			
Основные	Дополнительные ?	Динамика запросов ?	
Запрос	Кластер ?	Конкурентность ?	Спрос ?
crm для фитнеса	crm для фитнеса	● умеренная	557
crm для фитнес клуба	crm для фитнеса	● умеренная	423
яндекс маркет промокод crm для фитнеса	промокод яндекс маркет на первый заказ 	● низкая	220
crm для фитнес студии	crm для фитнеса	● умеренная	225
crm система для фитнес клуба	crm для фитнеса	● умеренная	126
crm для фитнес клуба dzen.ru/a/ai48xzipauup2w0n	crm для фитнес клуба dzen.ru/a/ai48xzipauup2w0n	● высокая	58
crm системы для фитнеса	crm для фитнеса	● умеренная	49
crm для фитнес студии dzen.ru/a/ajhggftjnfdsnloi	crm для фитнес студии dzen.ru/a/ajhggftjnfdsnloi	● высокая	58
crm системы для фитнес клуба	crm для фитнеса	● умеренная	45
crm система для фитнес студии	crm для фитнеса	● умеренная	40
crm для фитнес центра	crm для фитнеса	● умеренная	34
crm система для фитнеса	crm для фитнеса	● умеренная	30
crm для фитнес клуба dzen	crm для фитнес клуба dzen.ru/a/ai48xzipauup2w0n	● высокая	40
crm для фитнеса dzen.ru/a/ajc0f4uh9bk64b8g	crm для фитнеса dzen.ru/a/ajc0f4uh9bk64b8g	● высокая	19
crm для фитнес студии dzen	crm для фитнеса	● высокая	51

Рисунок 1 - Статистика поискового запроса в Яндекс.Вебмастере [1]

Для достижения цели необходимо решить следующие задачи:

1. проанализировать предметную область фитнес-клуба;
2. определить основные роли пользователей и функциональные требования;
3. спроектировать структуру приложения и схему базы данных;
4. реализовать приложение с использованием Python и Django;
5. настроить административный интерфейс для управления данными;
6. реализовать основные модели предметной области;
7. провести тестирование базовой логики приложения;
8. определить направления дальнейшего развития системы.

В качестве технологической основы выбран язык программирования Python и фреймворк Django. Django подходит для данной задачи, потому что предоставляет готовые средства для работы с моделями, базой данных, миграциями, маршрутизацией, шаблонами и административной панелью. Это позволило быстро получить рабочую систему и сосредоточиться на структуре данных и логике предметной области.

На этапе разработки используется SQLite. Эта база данных удобна для локальной разработки и не требует отдельной настройки сервера. При дальнейшем развитии проекта планируется переход на PostgreSQL, так как эта СУБД лучше подходит для более серьезной эксплуатации, расширенного поиска и масштабирования.

1 Анализ CRM систем для фитнес-клуба

Основная особенность данной области заключается в том, что внутри одной системы необходимо одновременно учитывать несколько взаимосвязанных сущностей: клиентов, тренеров, услуги, занятия по расписанию, записи на занятия, разовые посещения и абонементы.

1.1 Анализ деятельности фитнес-клуба

С практической точки зрения работа фитнес-клуба состоит не из одной операции, а из набора повторяющихся процессов. К ним относятся регистрация нового клиента, хранение и актуализация его контактных данных, создание перечня услуг, назначение тренеров, формирование расписания, оформление записи на занятие, учет разовых посещений и контроль срока действия абонементов. Даже при небольшом количестве клиентов эти процессы тесно связаны друг с другом. Например, запись клиента на занятие зависит от существования самого клиента в базе, наличия подходящей услуги, созданного занятия по расписанию, свободных мест и, в ряде случаев, действующего абонемента.

Основная проблема заключается в том, что данные и процессы часто оказываются разрозненными. Если информация о клиентах хранится отдельно от расписания, а записи на занятия ведутся вручную или в разных таблицах, то повышается вероятность дублирования данных, появления несогласованностей и ошибок при обслуживании клиентов. Особенно это заметно в ситуациях, когда необходимо быстро ответить на практические вопросы: кто записан на конкретное занятие, сколько осталось мест, есть ли у клиента действующий абонемент, какие тренеры свободны в определенное время, какие услуги сейчас активны.

Дополнительная сложность связана с тем, что фитнес-клуб обычно сочетает несколько форматов оказания услуг. В одном случае клиент покупает разовое посещение тренажерного зала, в другом - записывается на групповое занятие, в третьем - приобретает абонемент на определенный срок или число посещений. Следовательно, система должна поддерживать как разовые операции, так и длительные отношения с клиентом, связанные с абонементом и повторными посещениями.

1.2 Выявление ролей пользователей и функциональных требований

Для проектируемой системы можно выделить три основные роли пользователей.

Первая роль - **администратор**. Администратор отвечает за ведение клиентов, тренеров, услуг, расписания, записей и абонементов. Именно этот пользователь вносит основную часть данных в систему и обеспечивает их актуальность.

Вторая роль - **тренер**. Для тренера важна информация о собственных занятиях, рабочем времени и клиентах, записанных на тренировку. В более полном варианте системы тренер может просматривать свое расписание и список клиентов, однако в минимальной версии проекта основное управление остается за администратором.

Третья роль - **клиент**. Клиент является получателем услуг фитнес-клуба. В рамках текущей версии проекта клиент не управляет системой напрямую через полноценный личный кабинет, однако все ключевые действия, связанные с ним, должны поддерживаться: регистрация, запись на занятие, оформление разового посещения, оформление и учет абонемента.

Исходя из выделенных ролей, можно сформулировать основные функциональные требования к системе в виде пользовательских историй.

Пользовательские истории администратора:

1. Как администратор, я хочу добавлять, редактировать и деактивировать клиентов, чтобы поддерживать актуальную клиентскую базу.
2. Как администратор, я хочу добавлять и редактировать тренеров, чтобы хранить информацию о сотрудниках клуба.
3. Как администратор, я хочу создавать и изменять услуги, чтобы система содержала актуальный перечень доступных тренировок.
4. Как администратор, я хочу создавать занятия по расписанию с указанием тренера, времени, стоимости и количества мест, чтобы формировать календарь работы клуба.
5. Как администратор, я хочу записывать клиента на конкретное занятие, чтобы фиксировать его участие и не терять информацию о записи.
6. Как администратор, я хочу учитывать разовые посещения, чтобы система отражала посещения клиентов без оформления абонемента.
7. Как администратор, я хочу оформлять клиенту абонемент и отслеживать его срок действия, чтобы контролировать долгосрочные услуги клуба.

8. Как администратор, я хочу видеть список записей на занятие, чтобы понимать загрузку тренеров и доступность мест.

Пользовательские истории тренера:

1. Как тренер, я хочу видеть список своих занятий, чтобы понимать собственное расписание.
2. Как тренер, я хочу видеть список клиентов, записанных на занятие, чтобы подготовиться к тренировке.
3. Как тренер, я хочу видеть свои рабочие интервалы, чтобы контролировать доступное время.

Пользовательские истории клиента:

1. Как клиент, я хочу быть зарегистрированным в системе, чтобы сотрудники клуба могли быстро оформить мои посещения и записи.
2. Как клиент, я хочу записываться на занятие через сотрудника клуба, чтобы получить место на нужной тренировке.
3. Как клиент, я хочу приобретать абонемент или разовое посещение, чтобы пользоваться услугами клуба в удобном формате.

На основе этих пользовательских историй можно выделить основные функции системы: ведение клиентской базы, ведение базы тренеров, управление услугами, формирование расписания занятий, учет записей, учет разовых посещений и управление абонементом. Таким образом, разрабатываемая система должна решать не одну локальную задачу, а обеспечивать базовую автоматизацию ключевых процессов фитнес-клуба.

Такая структура приводит к использованию нескольких взаимосвязанных сущностей. Клиент связан с абонементом, разовыми посещениями и записями. Тренер связан с рабочими слотами и занятиями. Услуга определяет тип тренировки, ее длительность и стоимость. Занятие является конкретной реализацией услуги в определенную дату и время. Запись связывает клиента и занятие. Такая модель хорошо соответствует реляционной базе данных и позволяет построить понятную систему зависимостей между объектами.

Таким образом, для фитнес-клуба критически важны централизованное хранение данных, согласованность справочников и удобная работа с расписанием и посещениями.

1.1 Анализ существующих аналогов ПО

Для обоснования необходимости разработки собственной системы целесообразно рассмотреть несколько классов существующих решений, которые потенциально могут использоваться для автоматизации работы фитнес-клуба: электронные таблицы, универсальные CRM-системы и специализированные системы для фитнес-индустрии.

Первый возможный вариант - использование электронных таблиц, например Microsoft Excel. Excel предоставляет табличное хранение данных, средства сортировки, фильтрации и совместной работы над документом. Microsoft также поддерживает совместное редактирование книг несколькими пользователями. Однако электронная таблица остается универсальным инструментом для работы с данными, а не специализированной системой. В ней отсутствуют готовые сущности фитнес-клуба, такие как занятие, запись, абонемент, рабочий слот тренера и контроль вместимости занятия. В результате при росте объема данных пользователь вынужден самостоятельно поддерживать связи между таблицами и следить за целостностью информации, что снижает удобство и надежность такого решения [2].

Второй вариант - применение универсальной CRM-системы. В качестве примера можно рассмотреть amoCRM / Битрикс24 [3][12]. На официальных страницах продукт позиционируется как CRM для продаж и коммуникаций, в которой основное внимание уделяется чатам, воронкам продаж, автоматизации обработки лидов, поиску, фильтрации и встроенным коммуникационным каналам. Такой подход хорошо подходит для отделов продаж и клиентских коммуникаций, однако он изначально ориентирован на лиды, сделки и этапы воронки. Для фитнес-клуба этого недостаточно, так как здесь ключевыми сущностями выступают не сделки, а тренеры, занятия, записи, расписание, абонементы и посещения. Следовательно, универсальная CRM может быть использована только после заметной адаптации под конкретную область, что усложняет внедрение и делает решение компромиссным.

Третий вариант - специализированные системы для фитнес-клубов. К таким решениям относятся, например, mobifitness и fitness1c [4][9]. Mobifitness на официальном сайте предлагает инструменты для управления расписанием, онлайн-бронированием, платежами, отчетностью, работой сотрудников и клиентскими сервисами. Это делает систему значительно ближе к реальным потребностям фитнес-клуба по сравнению с универсальной CRM. Вместе с тем такая платформа является крупным коммерческим продуктом, ориентированным на широкий круг задач, и для небольшого проекта может оказаться избыточной по сложности и стоимости.

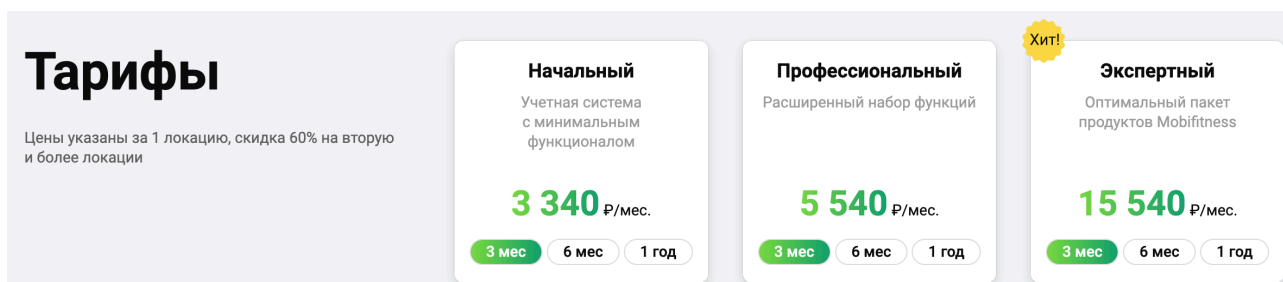


Рисунок 2 - Стоимость услуг Mobifitness за одну локацию на один месяц

Сравнение рассмотренных подходов показывает следующее. Электронные таблицы просты на старте, но плохо подходят для согласованного учета взаимосвязанных сущностей. Универсальные CRM-системы обладают развитой инфраструктурой работы с клиентами и коммуникациями, но недостаточно точно отражают специфику фитнес-клуба. Специализированные платформы наиболее близки, однако чаще всего являются коммерческими решениями с заранее заданной логикой и не всегда обеспечивают нужную гибкость. В этой связи разработка собственной системы представляется обоснованной.

2 Проектирование системы

2.1 Модульная структура приложения

При проектировании использовался модульный подход. Система была разделена на несколько логически независимых частей: `core`, `clients`, `trainers`, `services` и `bookings`. Модуль `core` отвечает за общие элементы проекта и публичную часть приложения. Модуль `clients` содержит сущности и операции, связанные с клиентами и их абонементом. Модуль `trainers` предназначен для хранения информации о тренерах и их рабочих слотах. Модуль `services` описывает услуги и планы абонементов. Модуль `bookings` объединяет сущности, связанные с занятиями, записями клиентов и разовыми посещениями. Такое разбиение позволило уменьшить связанность компонентов, сделать структуру проекта более понятной и упростить дальнейшее расширение системы.

- `clients` — хранит клиентов клуба.
- `trainers` — хранит тренеров.
- `services` — хранит виды тренировок: индивидуальные и групповые.
- `membership_plans` — хранит шаблоны абонементов: цена, срок, лимит посещений.
- `client_memberships` — хранит абонементы, которые купили конкретные клиенты.
- `trainer_work_slots` — хранит рабочие интервалы тренеров, чтобы видеть, когда они доступны для записи.
- `sessions` — хранит конкретные тренировки на дату и время с конкретным тренером.
- `bookings` — хранит записи клиентов на тренировки.
- `one_time_visits` — хранит разовые посещения зала без тренировки.

Рисунок 3 - Таблицы в БД

2.2 Объектно-ориентированное проектирование

С точки зрения объектно-ориентированного анализа и проектирования были выделены основные классы системы: `Client`, `Trainer`, `Service`, `MembershipPlan`, `ClientMembership`, `TrainerWorkSlot`, `Session`, `Booking` и `OneTimeVisit`. Каждый из этих классов отражает отдельную сущность и содержит только те атрибуты и операции, которые непосредственно относятся к его ответственности. Например, класс `Client` отвечает за хранение персональных данных клиента, класс `Trainer` - за данные тренера, класс `Session` - за конкретное занятие в расписании, а класс `Booking` - за факт записи клиента на выбранное занятие. Такой подход позволил сделать модель системы прозрачной и хорошо согласованной с будущей реляционной базой данных.

2.3 Проектирование базы данных

Итоговая схема не появилась сразу в окончательном виде. В процессе работы было подготовлено **пять версий базы данных**, каждая из которых уточняла состав сущностей и связи между ними. На ранних этапах структура была более простой и охватывала только клиентов, тренеров, услуги и записи. Однако по мере анализа предметной области стало ясно, что для корректного отражения логики фитнес-клуба необходимо отдельно выделить занятия по расписанию, планы абонементов, абонементы клиентов, рабочие интервалы тренеров и разовые посещения. В результате итоговая версия базы данных стала более полной и лучше соответствующей реальным бизнес-процессам.

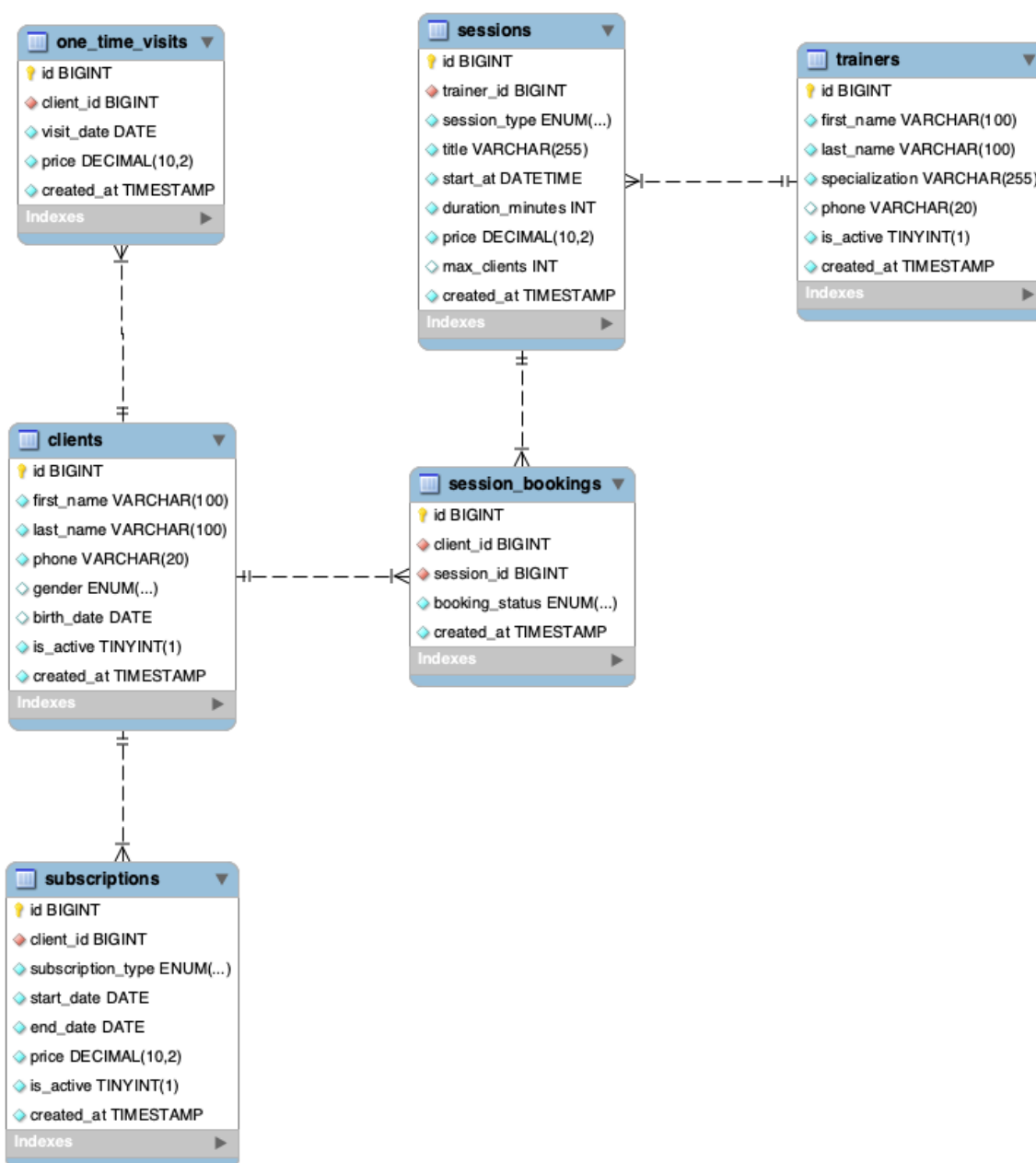


Рисунок 4 - Первая версия схемы базы данных

На каждом новом шаге проверялось отсутствие дублирования данных, корректность связей между сущностями и удобство последующей реализации средствами Django ORM. В итоговом варианте были сформированы следующие основные таблицы: `clients`, `trainers`, `services`, `membership_plans`, `client_memberships`, `trainer_work_slots`, `sessions`, `bookings` и `one_time_visits`.

При проектировании схемы базы данных также учитывался практический опыт использования программных решений в сфере услуг. Кроме собственных идей и анализа предметной области, были изучены открытые примеры проектирования подобных систем. В качестве ориентира использовались опубликованные в интернете примеры схем баз данных фитнес-клубов, в том числе открытая визуальная схема, размещенная на Reddit [5]. Изучение таких примеров позволило сравнить различные подходы к выделению сущностей и связей, а также проверить, насколько выбранная структура соответствует распространенной практике проектирования систем для фитнес-индустрии.

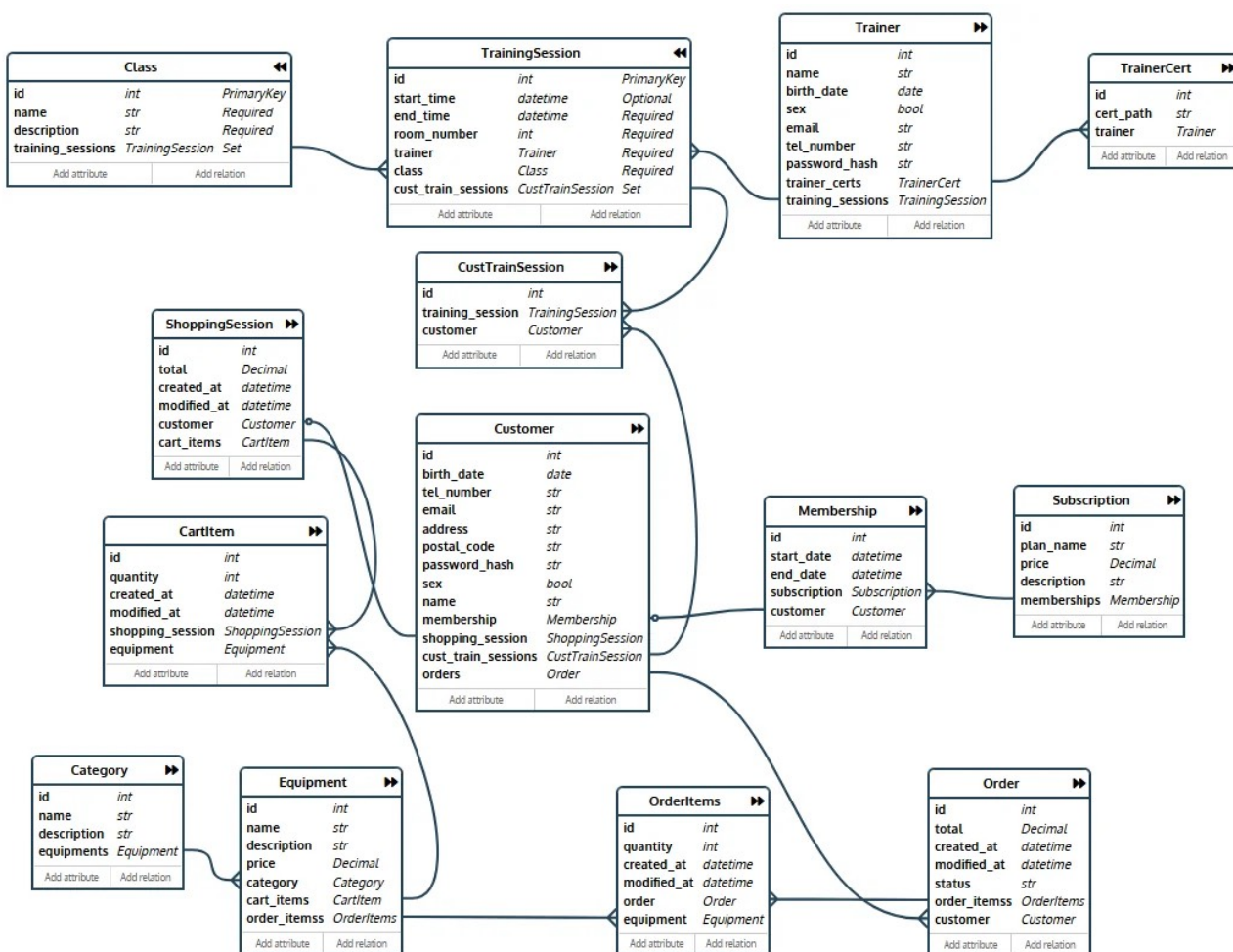


Рисунок 5 - Пример схемы базы данных фитнес-клуба с сайта Reddit

Результатом этапа проектирования стали две ключевые модели представления системы. Первая - **схема модулей**, описывающая основные сущности программной системы, их атрибуты и связи на уровне объектной модели. Вторая - **схема базы данных**, отражающая структуру хранения данных в реляционной форме. Схема модулей использовалась для описания логики приложения на уровне Python-моделей и будущих объектов Django ORM, а схема базы данных - для проектирования таблиц, внешних ключей, ограничений и индексов.

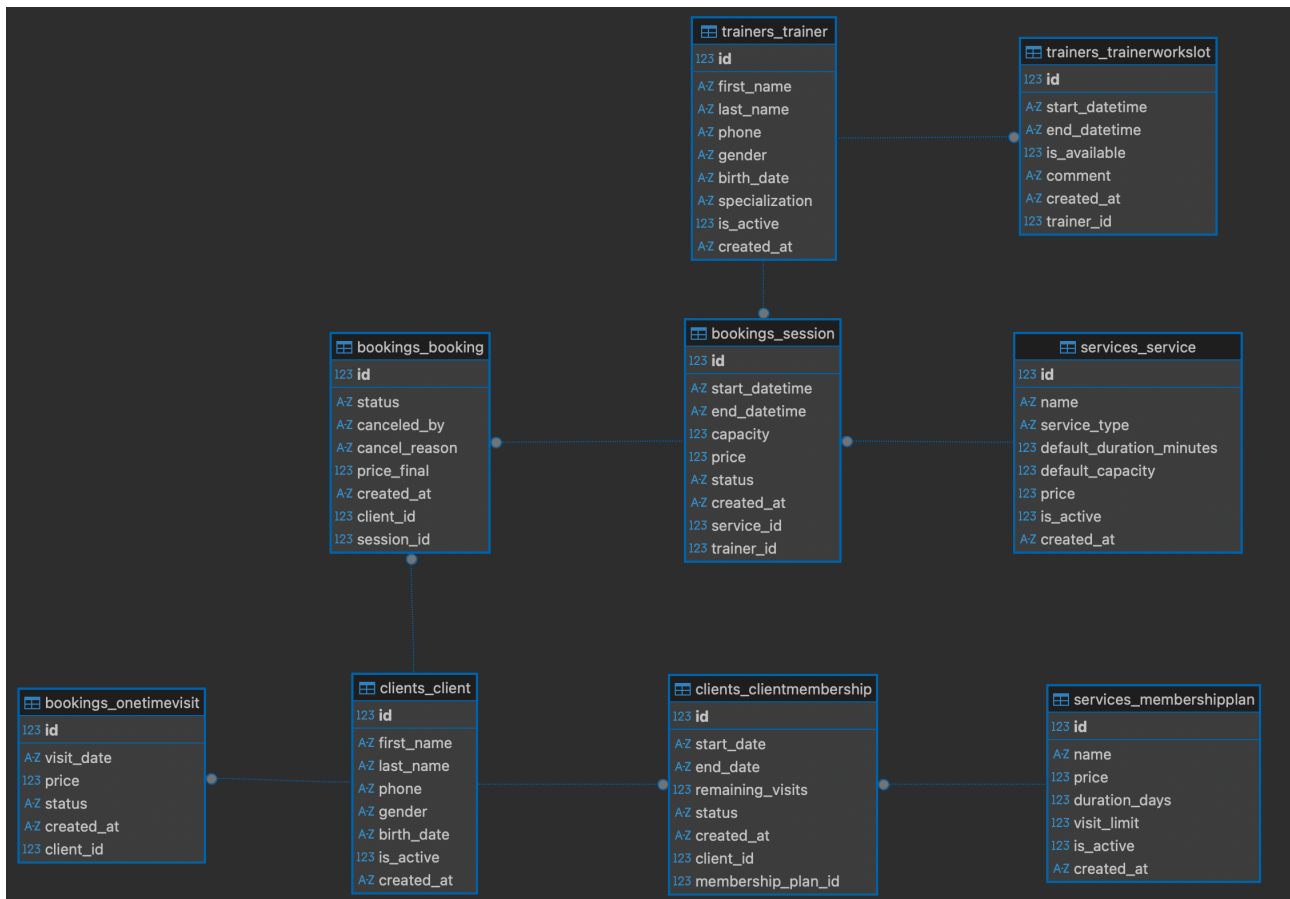


Рисунок 6 - Итоговая версия схемы базы данных приложения

Таким образом, этап проектирования стал основой всей дальнейшей разработки. На этом этапе была определена архитектура системы, зафиксированы ключевые сущности предметной области, выполнено разбиение на модули, последовательно проработана структура базы данных и подготовлены диаграммы.

3 Описание этапа кодирования

Основной задачей этапа кодирования являлось создание работающего приложения, в котором реализованы модели, административный интерфейс, публичная главная страница, механизмы хранения данных и базовые проверки корректности работы системы.

3.1 Обоснование выбора типа приложения

Для реализации системы был выбран тип **веб-приложения**. Такой вариант является наиболее подходящим для рассматриваемой предметной области, поскольку сотрудники фитнес-клуба могут получать доступ к системе через браузер без установки отдельной программы на каждый компьютер. Веб-приложение также удобно развивать: оно может быть размещено на сервере, подключено к единой базе данных и использовано несколькими сотрудниками одновременно.

Альтернативным вариантом могла быть разработка настольного приложения, однако такой подход имеет ряд ограничений. Настольная программа требует установки на конкретное устройство, сложнее обновляется и менее удобна при работе нескольких пользователей с единой базой данных. Мобильное приложение также не было выбрано в качестве основной формы реализации, поскольку для сотрудников фитнес-клуба важнее полноценный интерфейс управления данными, расписанием, клиентами и записями, что удобнее выполнять через веб-интерфейс.

Таким образом, веб-приложение является наиболее рациональным вариантом для данной системы, так как обеспечивает централизованное хранение данных, удобный доступ через браузер и возможность дальнейшего расширения функциональности.

3.2 Обоснование выбора технологий

В качестве основного языка программирования был выбран **Python**. Данный язык широко применяется для разработки веб-приложений, имеет понятный синтаксис и большое количество готовых библиотек.

Для разработки серверной части был выбран веб-фреймворк **Django**. Он предоставляет готовые средства для создания веб-приложений: маршрутизацию, работу с шаблонами, ORM для взаимодействия с базой данных, систему миграций, встроенную административную панель, механизмы авторизации и разграничения прав доступа. Для данного проекта особенно важным преимуществом Django является наличие встроенной админки, так как она позволяет быстро реализовать рабочий интерфейс для управления клиентами, тренерами, услугами, занятиями, записями и абонентами.

Для хранения данных на этапе разработки была выбрана **SQLite**. Эта СУБД не требует отдельной установки и настройки сервера базы данных, поэтому хорошо подходит для учебной разработки и прототипирования. При этом структура приложения построена с использованием Django ORM, что позволяет в дальнейшем перейти на более мощную СУБД, например **PostgreSQL**, без полной переработки кода приложения. Такой подход делает систему более гибкой: SQLite используется для локальной разработки, а PostgreSQL может применяться при дальнейшем развитии и развертывании проекта.

Для клиентской части использовались HTML, CSS и JavaScript. Публичная главная страница была реализована на основе готового HTML-шаблона, который был адаптирован под тематику фитнес-клуба. Для административной части на текущем этапе используется стандартная административная панель Django, так как она уже содержит необходимые механизмы управления данными и позволяет сосредоточиться на бизнес-логике приложения.

3.3 Реализация структуры приложения

Проект был разделен на несколько Django-приложений, каждое из которых отвечает за отдельную часть системы:

- **core** - общие страницы проекта, включая главную страницу;
- **clients** - клиенты и абонементы клиентов;
- **trainers** - тренеры и рабочие интервалы тренеров;
- **services** - услуги и планы абонементов;
- **bookings** - занятия, записи клиентов и разовые посещения.

Такое разбиение позволило сделать структуру проекта более понятной и поддерживаемой. Каждый модуль отвечает за собственную часть, что упрощает дальнейшую доработку приложения.

В процессе кодирования были реализованы основные модели системы: **Client**, **Trainer**, **Service**, **MembershipPlan**, **ClientMembership**, **TrainerWorkSlot**, **Session**, **Booking** и **OneTimeVisit**. Эти модели соответствуют сущностям, выделенным на этапе проектирования, и используются Django ORM для создания таблиц базы данных и выполнения операций с данными.

Для моделей были описаны поля, связи между сущностями, человекочитаемые названия, сортировка, строковое представление объектов и базовая валидация. Например, для занятий и рабочих слотов тренеров реализована проверка, что время окончания не может быть раньше времени начала. Для записи клиента на

занятие задано ограничение уникальности, запрещающее записать одного и того же клиента на одно и то же занятие повторно.

После описания моделей были созданы и применены миграции базы данных. Затем модели были зарегистрированы в административной панели Django. Это позволило получить готовый интерфейс для добавления, изменения, поиска и фильтрации данных.

3.4 Клиент-серверная архитектура

Разработанное приложение имеет клиент-серверную архитектуру. Пользователь взаимодействует с системой через браузер. Запросы от браузера поступают на серверную часть приложения, реализованную на Django. Django обрабатывает запрос, при необходимости обращается к базе данных через ORM, получает или изменяет данные и возвращает пользователю HTML-страницу.

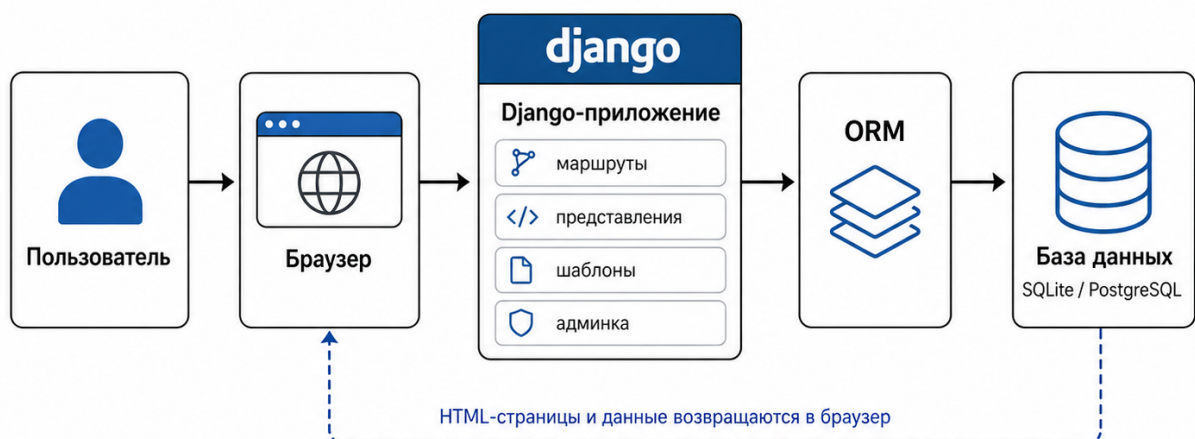


Рисунок 7 - Клиент-серверная архитектура приложения

В текущей версии приложения можно выделить следующие основные компоненты:

1. **Клиентский уровень** - браузер пользователя, в котором отображаются HTML-страницы.
2. **Уровень приложения** - Django-проект, содержащий маршруты, представления, модели и административную панель.
3. **Уровень данных** - база данных SQLite на этапе разработки.
4. **Административный интерфейс** - встроенная Django admin, используемая для управления сущностями системы.

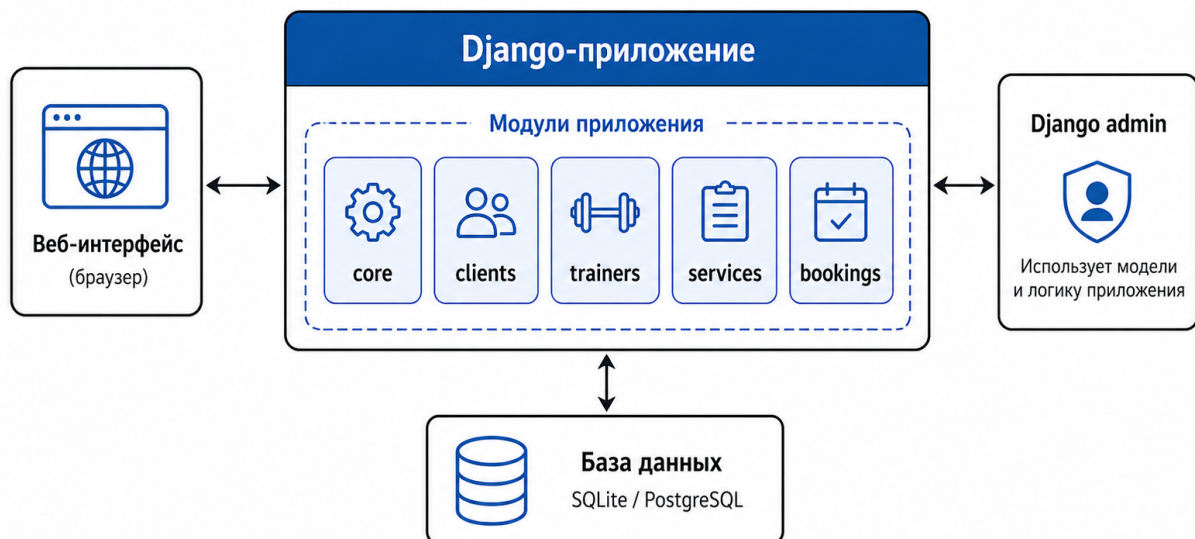


Рисунок 8 - Диаграмма компонентов приложения

4 Тестирование приложения

4.1 Описание тестируемых компонентов

Для тестирования использовались стандартные средства Django, основанные на модуле `django.test`. Были подготовлены тесты для основных приложений проекта:

- `clients` - тестирование клиента и абонеента клиента;
- `trainers` - тестирование тренера и рабочего слота тренера;
- `services` - тестирование услуг и планов абонементов;
- `bookings` - тестирование занятий, записей и разовых посещений.

4.2 Проверка моделей и бизнес-логики

В рамках тестирования проверялись следующие сценарии:

1. корректное создание клиента;
2. корректное формирование полного имени клиента;
3. создание тренера и получение его полного имени;
4. создание услуги и плана абонеента;
5. создание абонеента клиента;
6. запрет некорректного периода действия абонеента;
7. создание рабочего слота тренера;
8. запрет рабочего слота, у которого окончание раньше начала;
9. создание занятия;
10. запрет занятия с некорректным временем окончания;
11. создание записи клиента на занятие;
12. запрет повторной записи одного клиента на одно и то же занятие;
13. создание разового посещения.

Для запуска тестов использовалась команда:

```
python manage.py test
```


5 Руководство пользователя

Разработанное приложение предназначено для автоматизации основных процессов фитнес-клуба. В текущей версии система состоит из публичной главной страницы и административной панели, через которую сотрудники могут управлять клиентами, тренерами, услугами, занятиями, записями, абонементом и разовыми посещениями.

5.1 Запуск приложения

Для запуска приложения необходимо открыть проект в среде разработки, активировать виртуальное окружение и выполнить команду запуска локального сервера:

```
python manage.py runserver
```

После запуска сервер становится доступен в браузере по адресу:

<http://127.0.0.1:8000/>

При переходе по данному адресу открывается главная страница приложения.

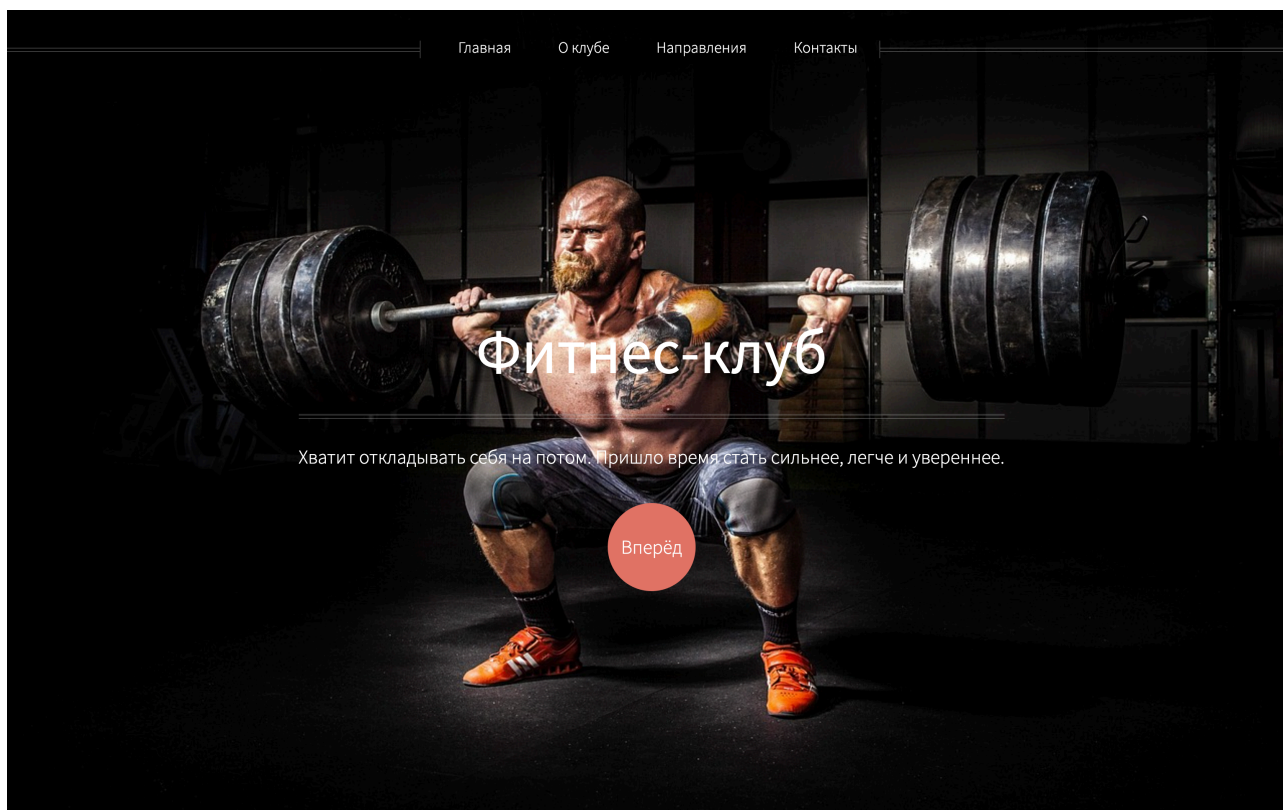


Рисунок 9 - Главная страница приложения

На главной странице отображается общая информация о фитнес-клубе, его услугах и назначении системы. Публичная часть приложения выполняет роль информационной страницы и демонстрирует внешний интерфейс проекта.

5.2 Вход в административную панель

Для управления данными используется административная панель Django. Чтобы открыть ее, необходимо перейти по адресу:

<http://127.0.0.1:8000/admin/>

После этого откроется форма авторизации. Пользователь должен ввести логин и пароль учетной записи администратора.

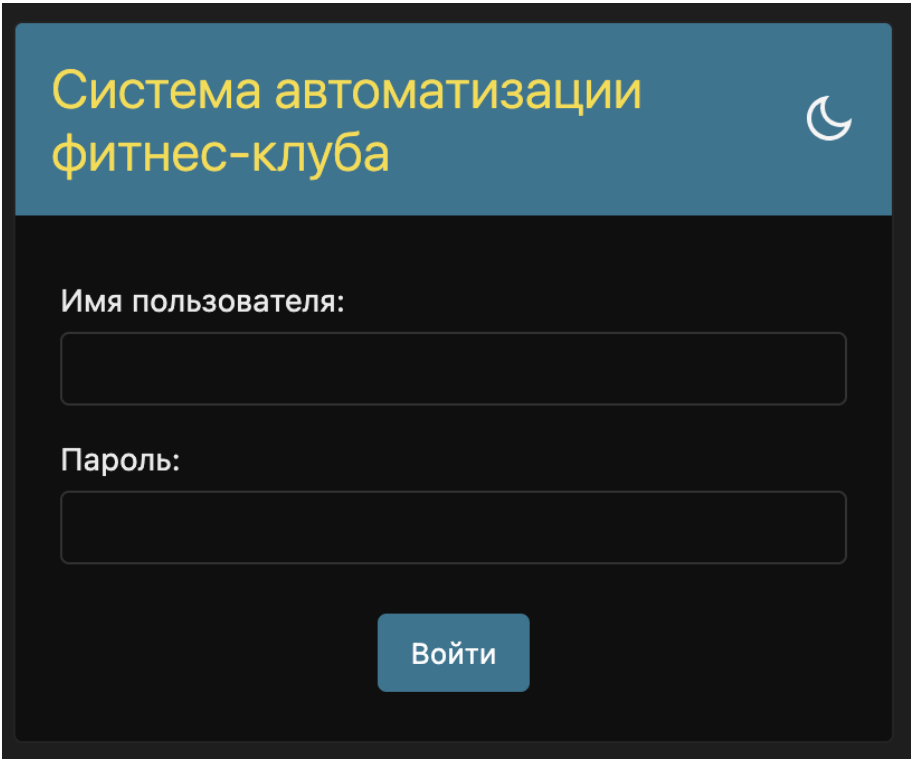
The image shows a web interface for a fitness club automation system. At the top, there is a blue header with the text "Система автоматизации фитнес-клуба" in yellow. To the right of the text is a white crescent moon icon. Below the header, the background is dark gray. There are two input fields: the first is labeled "Имя пользователя:" and the second is labeled "Пароль:". Below these fields is a blue button with the text "Войти" in white.

Рисунок 10 - Страница входа в административную панель

В системе предусмотрена учетная запись суперпользователя, который имеет полный доступ к управлению системой. Также может быть создан обычный администратор с ограниченными правами. Обычный администратор получает доступ к работе с основными сущностями системы, но не управляет пользователями и правами доступа.

5.3 Работа с клиентами

Для работы с клиентами необходимо открыть раздел **Клиенты**. В данном разделе отображается список клиентов фитнес-клуба. Для каждого клиента хранятся имя, фамилия, телефон, пол, дата рождения, статус активности и дата создания записи.

Администратор может:

- просматривать список клиентов;
- добавлять нового клиента;
- изменять данные существующего клиента;
- деактивировать клиента без удаления его из системы;
- искать клиента по имени, фамилии или телефону.

Система автоматизации фитнес-клуба

ДОБРО ПОЖАЛОВАТЬ, ИВАН. ОТКРЫТЬ САЙТ / ИЗМЕНИТЬ ПАРОЛЬ / ВЫЙТИ

Начало > Clients > Клиенты

Начните печатать для фильтрации...

BOOKINGS

- Занятия + Добавить
- Записи + Добавить
- Разовые посещения + Добавить

CLIENTS

- Абонементы клиентов + Добавить
- Клиенты + Добавить

SERVICES

- Планы абонементов + Добавить
- Услуги + Добавить

TRAINERS

- Рабочие слоты тренеров + Добавить
- Тренеры + Добавить

Выберите Клиент для изменения

Поиск: Найти

ID	ФАМИЛИЯ	ИМЯ	ТЕЛЕФОН	ПОЛ	ДАТА РОЖДЕНИЯ	АКТИВЕН	СОЗДАН
15	Беляева	Наталья	+79990000015	Женский	8 июня 1986 г.	✓	11 апреля 2026 г. 15:48
6	Васильев	Андрей	+79990000006	Мужской	25 июня 1993 г.	✓	11 апреля 2026 г. 15:48
9	Волков	Егор	+79990000009	Мужской	9 августа 1996 г.	✓	11 апреля 2026 г. 15:48
21	Иванов	Иван	+79990000021	Мужской	14 марта 1992 г.	✓	12 апреля 2026 г. 15:52
1	Иванов	Иван	+79990000001	Мужской	14 марта 1992 г.	✓	11 апреля 2026 г. 15:48
11	Козлова	Анна	+79990000011	Женский	12 мая 1994 г.	✓	11 апреля 2026 г. 15:48
17	Комарова	Юлия	+79990000017	Женский	23 ноября 1998 г.	✓	11 апреля 2026 г. 15:48
4	Кузнецов	Дмитрий	+79990000004	Мужской	5 ноября 1990 г.	✓	11 апреля 2026 г. 15:48
10	Лебедев	Роман	+79990000010	Мужской	17 февраля 1991 г.	✓	11 апреля 2026 г. 15:48
14	Михайлова	Ольга	+79990000014	Женский	19 января 1993 г.	✗	11 апреля 2026 г. 15:48
8	Морозов	Николай	+79990000008	Мужской	30 апреля 1985 г.	✗	11 апреля 2026 г. 15:48
19	Никитина	Ксения	+79990000019	Женский	11 декабря 1995 г.	✓	11 апреля 2026 г. 15:48
12	Новикова	Мария	+79990000012	Женский	3 октября 1989 г.	✓	11 апреля 2026 г. 15:48
18	Орлова	Светлана	+79990000018	Женский	6 апреля 1992 г.	✓	11 апреля 2026 г. 15:48
20	Павлова	Дарья	+79990000020	Женский	29 августа 1999 г.	✓	11 апреля 2026 г. 15:48
22	Петров	Пётр	+79990000022	Мужской	21 июля 1988 г.	✓	12 апреля 2026 г. 15:52
2	Петров	Пётр	+79990000002	Мужской	21 июля 1988 г.	✓	11 апреля 2026 г. 15:48
5	Попов	Сергей	+79990000005	Мужской	18 сентября 1987 г.	✓	11 апреля 2026 г. 15:48
3	Смирнов	Алексей	+79990000003	Мужской	10 января 1995 г.	✓	11 апреля 2026 г. 15:48
23	Смирнова	Анна	+79990000023	Женский	9 ноября 1995 г.	✓	12 апреля 2026 г. 15:52

ДОБАВИТЬ КЛИЕНТ +

ФИЛЬТР

- Показать счётчики
- Пол
 - Все
 - Мужской
 - Женский
- Активен
 - Все
 - Да
 - Нет
- Создан
 - Любая дата
 - Сегодня
 - Последние 7 дней
 - Этот месяц
 - Этот год

Рисунок 11 - Список клиентов в административной панели

Для добавления нового клиента необходимо нажать кнопку добавления записи, заполнить обязательные поля и сохранить объект. После сохранения клиент становится доступен для дальнейшей работы: оформления абонеента, записи на занятие или регистрации разового посещения.

5.4 Работа с тренерами

Раздел **Тренеры** предназначен для хранения информации о сотрудниках, проводящих занятия. Для каждого тренера указываются имя, фамилия, телефон, пол, дата рождения, специализация и статус активности.

Администратор может добавлять новых тренеров, редактировать их данные и отмечать тренера как неактивного. Это позволяет сохранить историю данных, не удаляя тренера из базы.

Также в системе предусмотрены рабочие слоты тренеров. Рабочий слот содержит тренера, дату и время начала, дату и время окончания, признак доступности и комментарий. Эта сущность нужна для дальнейшего развития расписания и контроля доступного времени тренера.

5.5 Работа с услугами и абонеентами

В разделе **Услуги** администратор управляет перечнем услуг фитнес-клуба. Услуга содержит название, тип, длительность по умолчанию, вместимость по умолчанию, стоимость и статус активности.

В системе предусмотрены два типа услуг:

- групповое занятие;
- индивидуальное занятие.

Для абонеентов используется отдельный раздел **Планы абонеентов**. План абонеента описывает не конкретную покупку клиента, а тип абонеента, который может быть выдан разным клиентам. В нем указываются название, стоимость, срок действия в днях и лимит посещений. Если лимит посещений не указан, абонеент может рассматриваться как безлимитный в рамках заданного срока.

5.6 Оформление абонеента клиента

После создания плана абонеента администратор может оформить абонеент конкретному клиенту. Для этого используется раздел **Абонеенты клиентов**. В записи указывается клиент, выбранный план абонеента, дата начала, дата окончания, количество оставшихся посещений и статус.

Статусы абонементов:

- активен;
- истек;
- заморожен;
- израсходован.

При создании абонементов система проверяет корректность дат: дата окончания не должна быть раньше даты начала. Это позволяет избежать ошибочных записей.

5.7 Работа с занятиями и записями

Раздел **Занятия** используется для формирования расписания. Занятие связывает услугу, тренера, дату и время проведения, количество мест, стоимость и статус.

Статусы занятия:

- запланировано;
- проведено;
- отменено.

При создании занятия система проверяет, что время окончания больше времени начала. Это предотвращает создание некорректных записей в расписании.

После создания занятия можно оформить запись клиента. Для этого используется раздел **Записи**. Запись связывает клиента и конкретное занятие. Также в записи указывается итоговая стоимость и статус.

Статусы записи:

- записан;
- посетил;
- отменил;
- не пришел.

В системе установлено ограничение, запрещающее повторно записать одного и того же клиента на одно и то же занятие. Это позволяет избежать дублирования записей.

5.8 Учет разовых посещений

Для клиентов, которые посещают зал без абонеента и без записи на конкретное занятие, предусмотрен раздел **Разовые посещения**. В записи указывается клиент, дата посещения, стоимость и статус.

Статусы разового посещения:

- оплачено;
- посетил;
- отменено.

Этот раздел позволяет учитывать клиентов, которые не являются постоянными посетителями, но пользуются услугами фитнес-клуба разово.

5.9 Завершение работы

После завершения работы с системой администратор может выйти из административной панели с помощью кнопки выхода. Это необходимо для защиты данных от несанкционированного доступа, особенно если приложение используется на общем компьютере.

Заключение

В ходе дипломной работы было разработано веб-приложение для автоматизации работы фитнес-клуба. Цель работы заключалась в создании системы, позволяющей вести учет клиентов, тренеров, услуг, занятий, записей, абонементов и разовых посещений. Данная цель была достигнута: приложение реализовано, основные сущности предметной области созданы, база данных спроектирована, административный интерфейс настроен, тестовые данные добавлены, а ключевая логика проверена с помощью тестов.

В процессе работы были решены основные задачи дипломного проекта. Был выполнен анализ предметной области, определены роли пользователей и основные функциональные требования. На этапе проектирования была разработана структура базы данных и модульная структура Django-проекта. Приложение было разделено на несколько логических модулей: `core`, `clients`, `trainers`, `services` и `bookings`. Такое разделение позволило сделать проект более понятным и удобным для дальнейшего развития.

Особое внимание было уделено проектированию базы данных. Итоговая структура была получена не сразу: в процессе работы было подготовлено несколько вариантов схемы, после чего была выбрана финальная версия. В нее вошли таблицы для хранения клиентов, тренеров, услуг, занятий, записей, планов абонементов, абонементов клиентов, рабочих слотов тренеров и разовых посещений. Такая структура позволяет достаточно полно описать основные процессы фитнес-клуба.

Для реализации был выбран язык Python и фреймворк Django. Этот выбор оказался удачным, так как Django предоставляет готовые средства для работы с моделями, миграциями, базой данных, административной панелью и авторизацией. Использование встроенной административной панели позволило быстро получить рабочий интерфейс для управления данными без необходимости сразу разрабатывать отдельную панель администратора.

В приложении реализованы основные операции: добавление и редактирование клиентов, тренеров, услуг, занятий, абонементов и записей. Также добавлены проверки корректности данных. Например, система не допускает создание занятия или рабочего слота с некорректным временем, а также запрещает повторную запись одного клиента на одно и то же занятие. Для проверки этой логики были написаны тесты моделей и ограничений.

Одной из особенностей работы является то, что проект может рассматриваться не только как учебное приложение, но и как основа для дальнейшего развития. В текущей версии реализован минимально необходимый функционал для учета данных фитнес-клуба. При этом структура приложения позволяет постепенно добавлять новые возможности без полной переработки проекта.

Не все возможности удалось реализовать в рамках текущей версии. В частности, при использовании SQLite возникло ограничение, связанное с поиском без учета регистра в административной панели, особенно для русскоязычных данных. В дальнейшем эту проблему планируется решить переходом на PostgreSQL, который лучше подходит для более серьезной эксплуатации, расширенного поиска и масштабирования приложения.

К перспективам развития проекта можно отнести создание отдельной панели администратора с использованием шаблона SB Admin [6]. Это позволит сделать более удобный интерфейс для сотрудников фитнес-клуба, не ограничиваясь стандартной Django admin. Также планируется добавить личный кабинет клиента, где пользователь сможет самостоятельно записываться на занятия, отменять или переносить запись, просматривать свое расписание и информацию об абонементе.

В дальнейшем проект может быть расширен до полноценной CRM-системы для фитнес-клубов и других организаций сферы услуг. При добавлении онлайн-записи, уведомлений, платежей, отчетов и гибкой настройки бизнес-процессов система может стать более конкурентоспособным решением и потенциально развиваться в направлении продуктов уровня Mobifitness или 1С:Фитнес клуб [9].

Таким образом была создана рабочая программная система, соответствующая поставленным целям и задачам.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Яндекс Вебмастер : официальный сайт [Электронный ресурс]. – URL: <https://webmaster.yandex.ru/> (дата обращения 28.03.2026).
2. Microsoft : официальный сайт [Электронный ресурс]. – URL: <https://www.microsoft.com/> (дата обращения 28.03.2026).
3. amoCRM : официальный сайт программного продукта [Электронный ресурс]. – URL: <https://www.amocrm.com/> (дата обращения 28.03.2026).
4. Mobifitness : официальный сайт программного продукта [Электронный ресурс]. – URL: <https://mobifitness.ru/> (дата обращения 28.03.2026).
5. Пример схемы базы данных фитнес-клуба : изображение [Электронный ресурс]. – URL: <https://www.reddit.com/media?url=https%3A%2F%2Fpreview.redd.it%2Ffitness-club-db-design-v0-4ietyhfdjcd1.png%3Fwidth%3D1058%26format%3Dpng%26auto%3Dwebp%26s%3D7daf7b329e894544610559db067500eabcf71bba> (дата обращения 02.04.2026).
6. SB Admin - Free Bootstrap Admin Template : шаблон административной панели [Электронный ресурс]. – URL: <https://startbootstrap.com/template/sb-admin> (дата обращения 15.04.2026).
7. Python : официальный сайт языка программирования [Электронный ресурс]. – URL: <https://www.python.org/> (дата обращения 25.03.2026).
8. Django : официальный сайт веб-фреймворка [Электронный ресурс]. – URL: <https://www.djangoproject.com/> (дата обращения 25.03.2026).
9. 1С:Фитнес клуб : официальный сайт программного продукта [Электронный ресурс]. – URL: <https://www.fitness1c.ru/> (дата обращения 28.03.2026).
10. SQLite : официальный сайт системы управления базами данных [Электронный ресурс]. – URL: <https://sqlite.org/index.html> (дата обращения 25.03.2026).
11. PostgreSQL : официальный сайт системы управления базами данных [Электронный ресурс]. – URL: <https://www.postgresql.org/> (дата обращения 15.04.2026).
12. Битрикс24 : официальный сайт программного продукта [Электронный ресурс]. – URL: <https://www.bitrix24.ru/> (дата обращения 28.03.2026).